Lecture 3: Processes

CS211 - Operating systems

Spring 2026

What happens when a program is compiled/run?

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

?

Executable
(e.g. a.out)

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

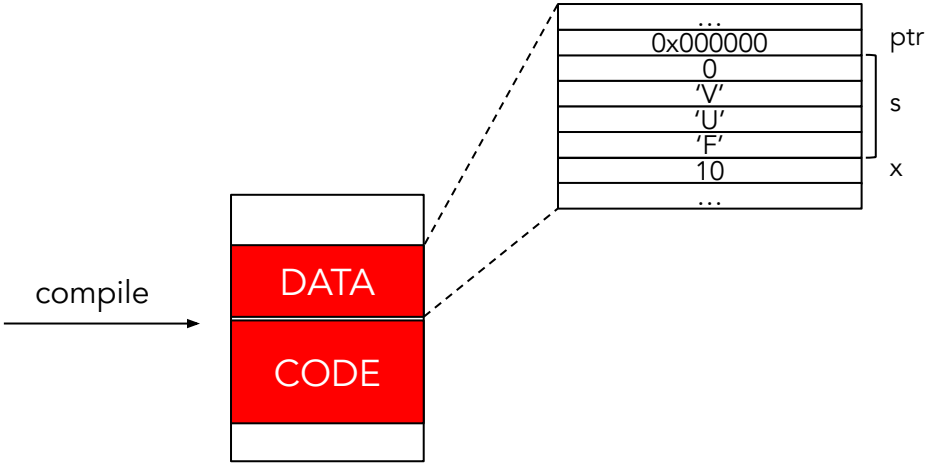
compile →

Executable
(e.g. a.out)

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)



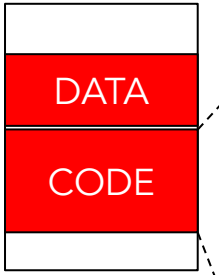
Executable
(e.g. a.out)

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →



Executable
(e.g. a.out)

```
main:
        pushq   %rbp
        movq    %rsp, %rbp
        …
        call    malloc@PLT
        movq    %rax, ptr(%rip)
        …
        call    f
        …
        leave
        ret
f:
        pushq   %rbp
        …
        call    free@PLT
        movl    $0, %eax
        call    g
        …
        ret
g:
        pushq   %rbp
        …
        ret
```

What happens when a program is compiled/run?

```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

Executable
(e.g. a.out)

run →

???

What happens when a program is compiled/run?

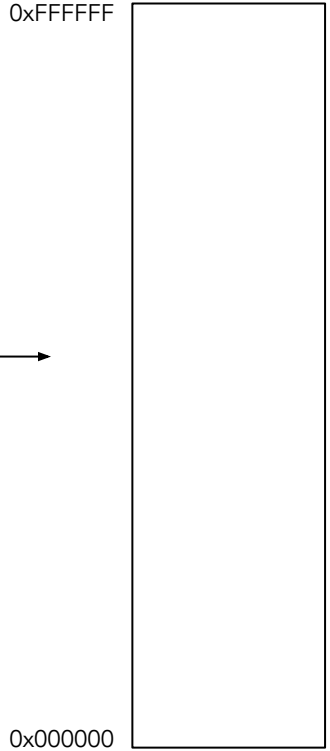
```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

Executable
(e.g. a.out)

run →



Memory

What happens when a program is compiled/run?

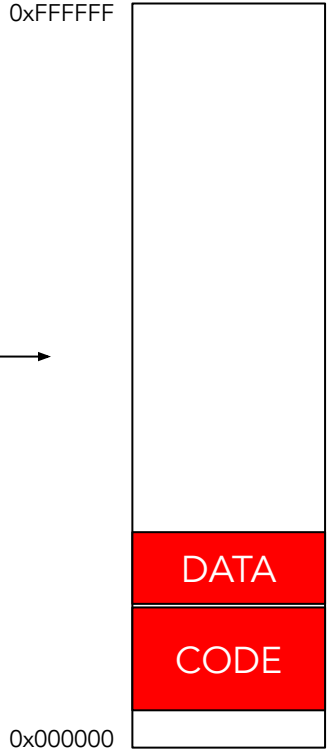
```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

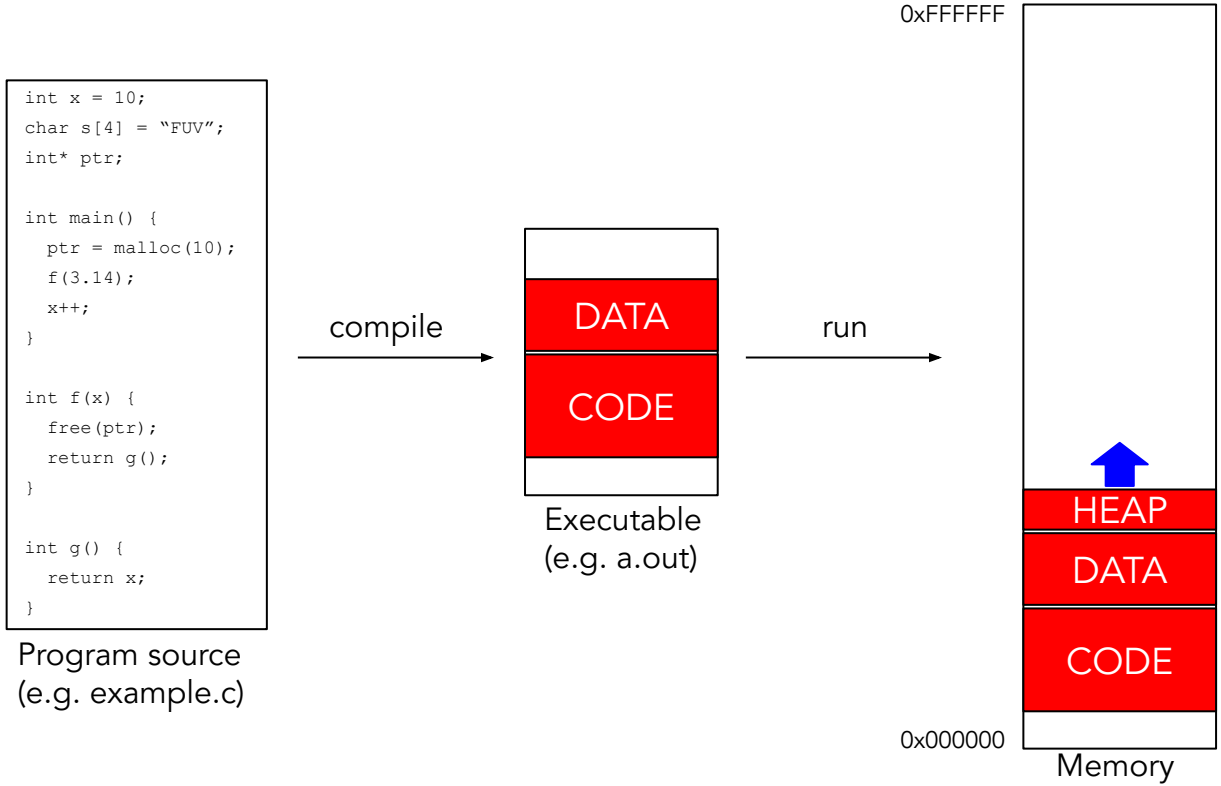
Executable
(e.g. a.out)

run →



Memory

What happens when a program is compiled/run?



What happens when a program is compiled/run?

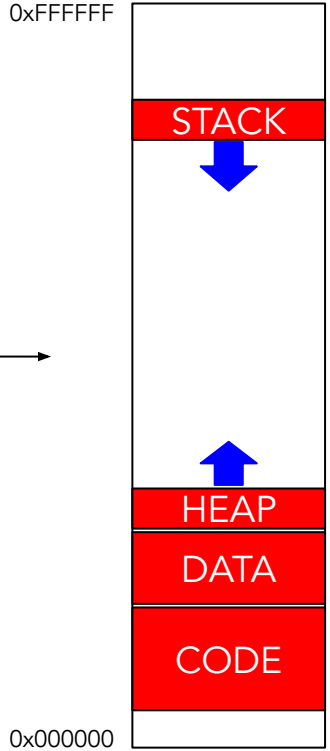
```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

Executable
(e.g. a.out)

run →



Memory

What happens when a program is compiled/run?

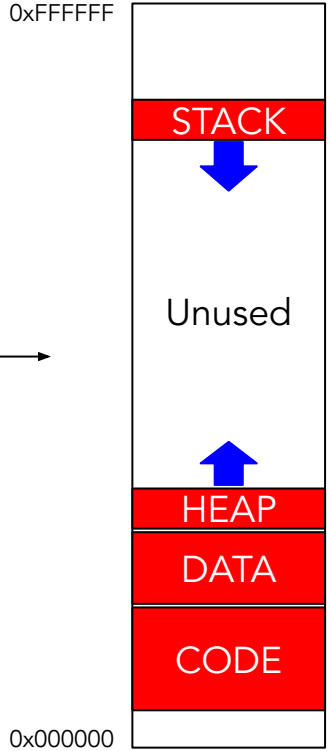
```
int x = 10;
char s[4] = "FUV";
int* ptr;

int main() {
  ptr = malloc(10);
  f(3.14);
  x++;
}

int f(x) {
  free(ptr);
  return g();
}

int g() {
  return x;
}
```

Program source
(e.g. example.c)

compile →

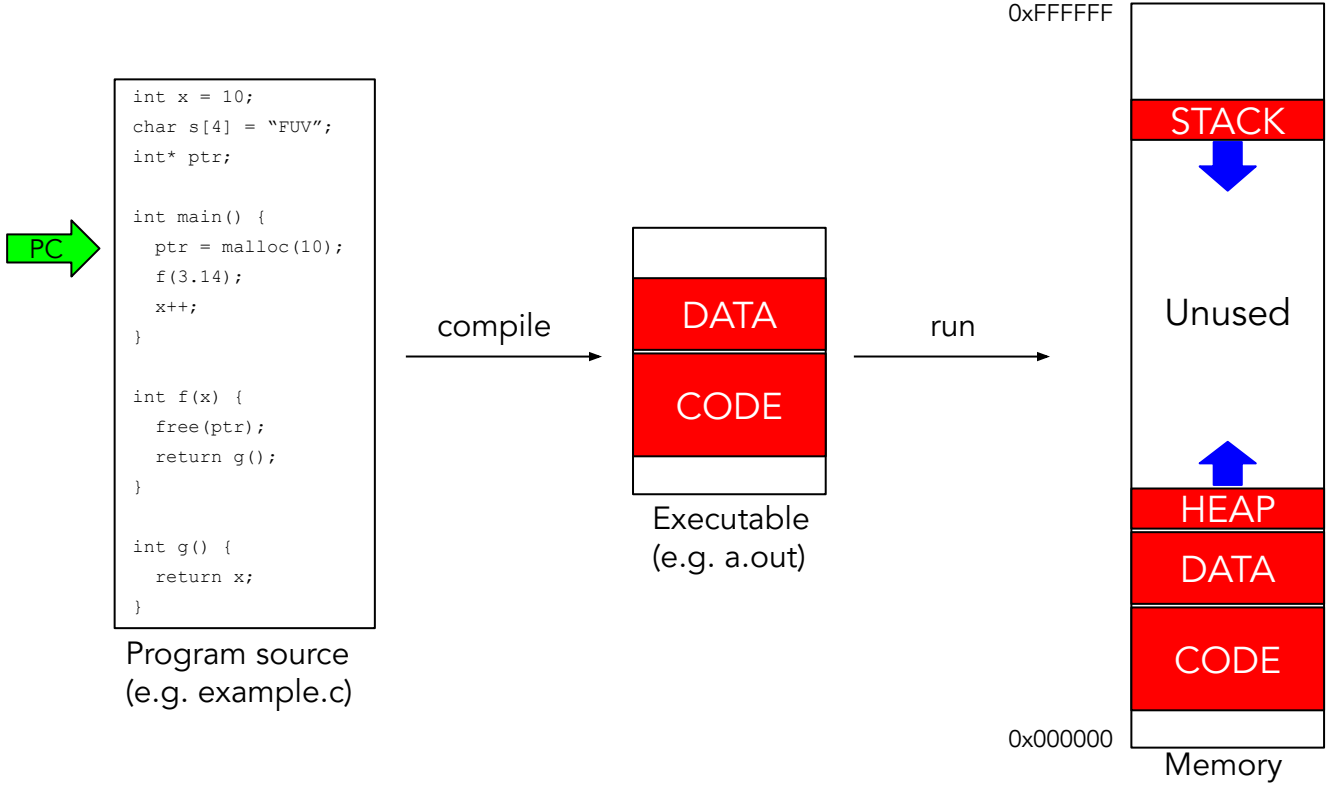
Executable
(e.g. a.out)

run →

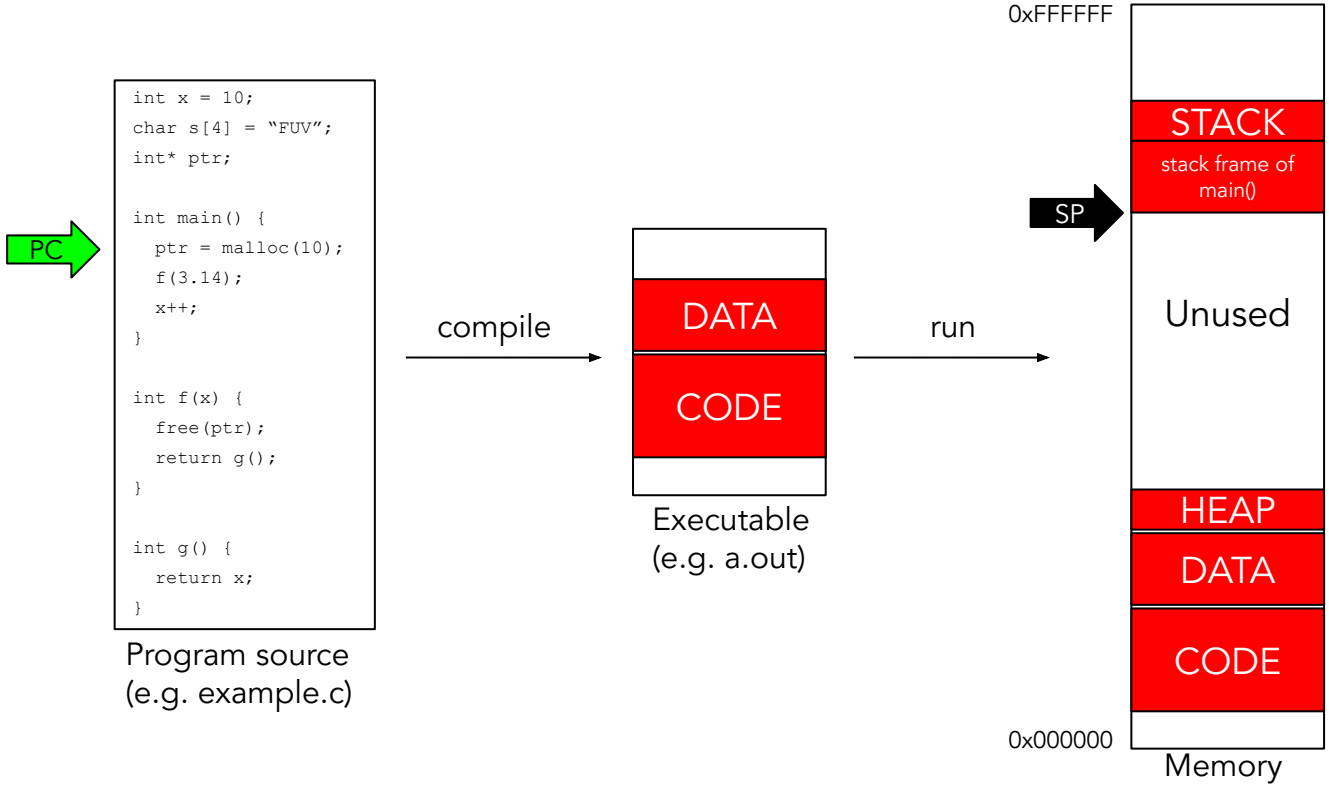


Memory

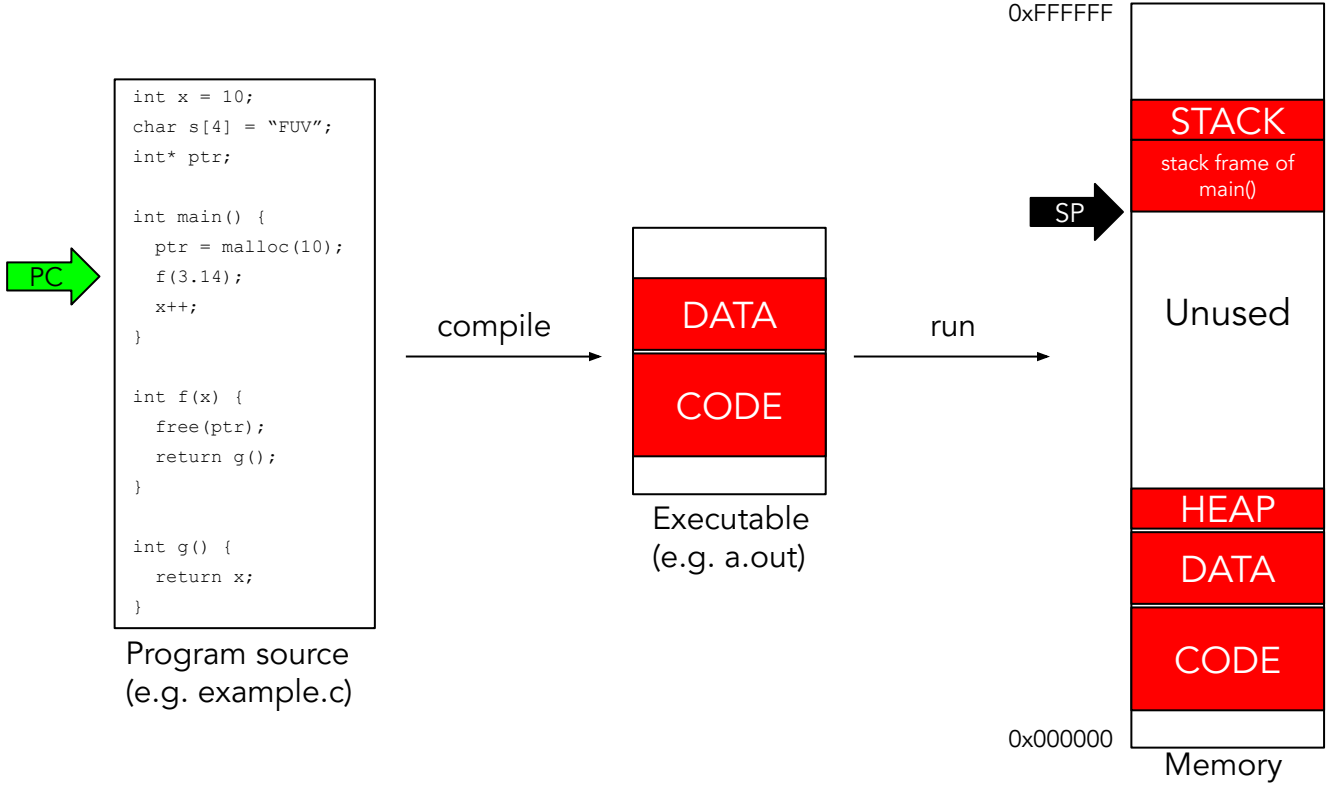
What happens when a program is compiled/run?



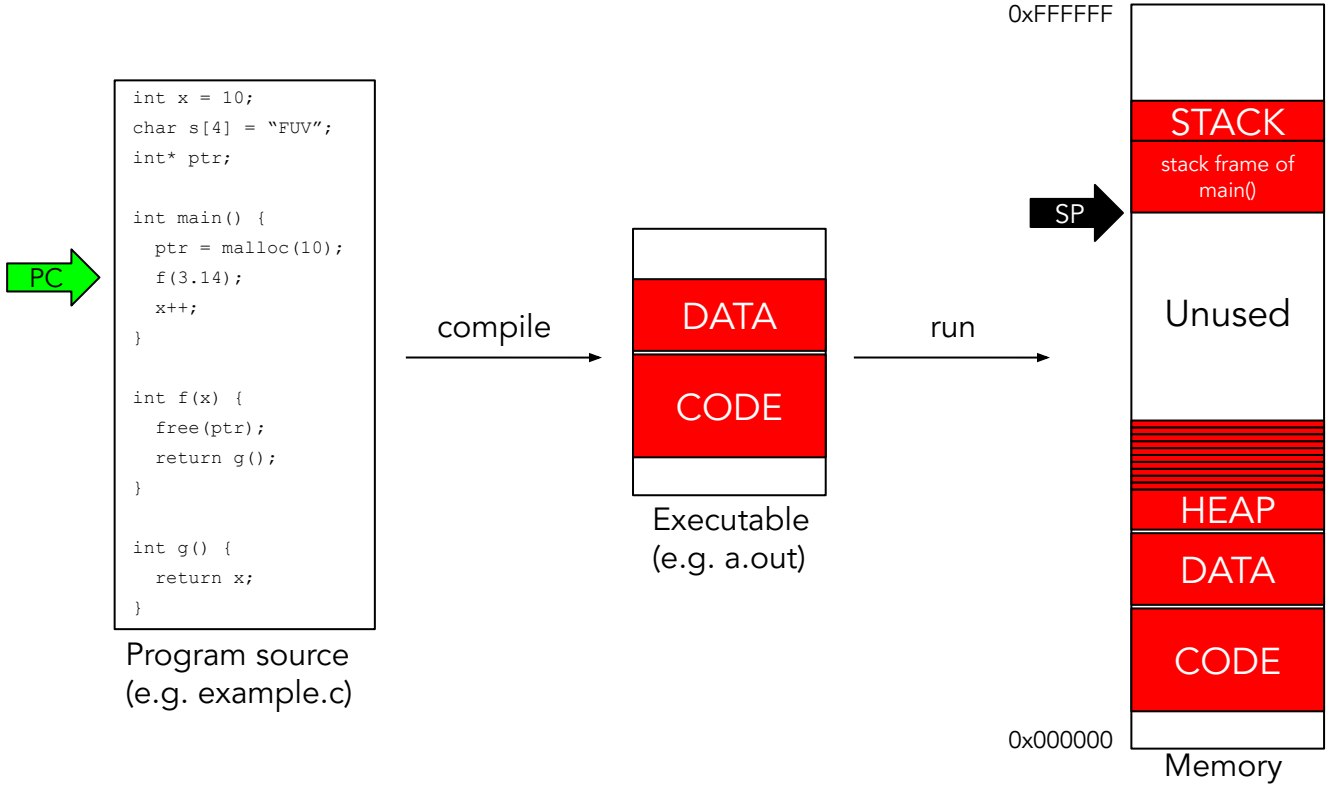
What happens when a program is compiled/run?



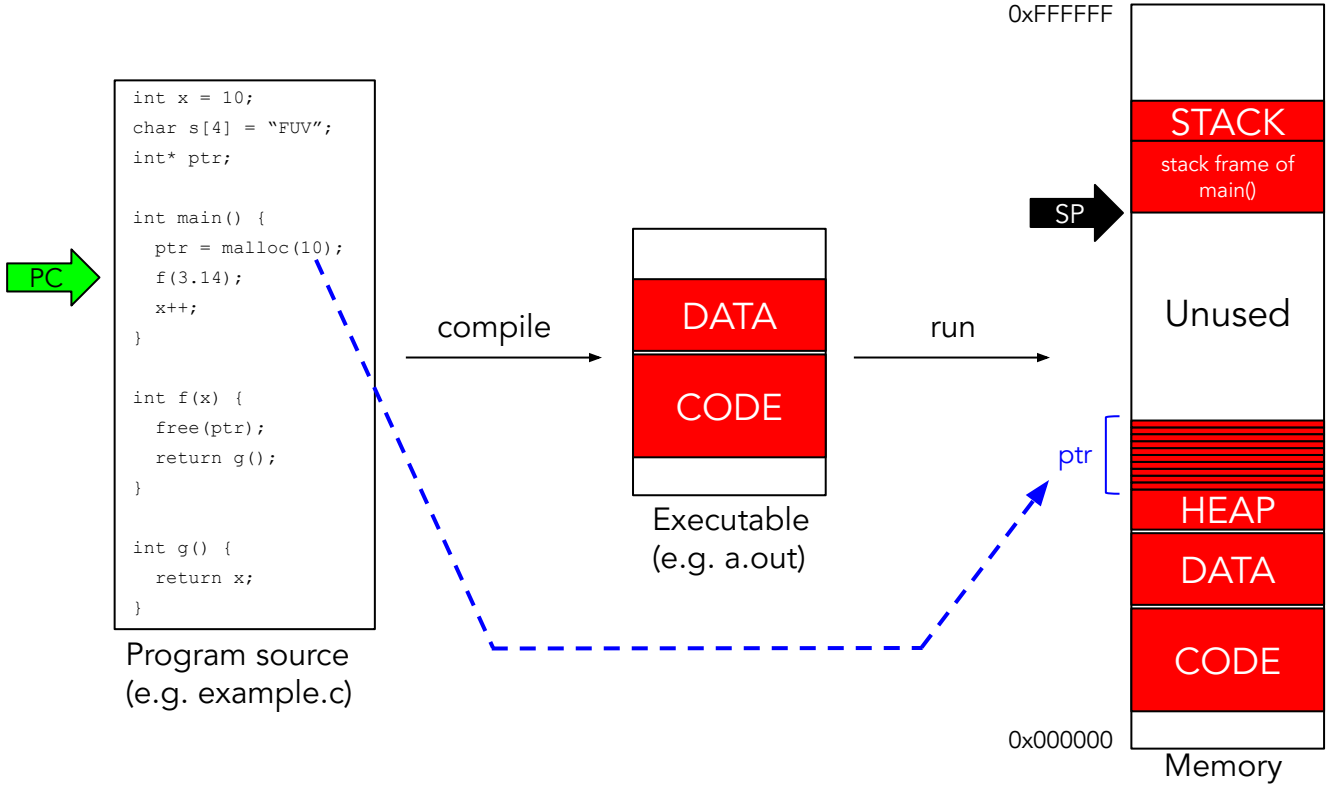
What happens when a program is compiled/run?



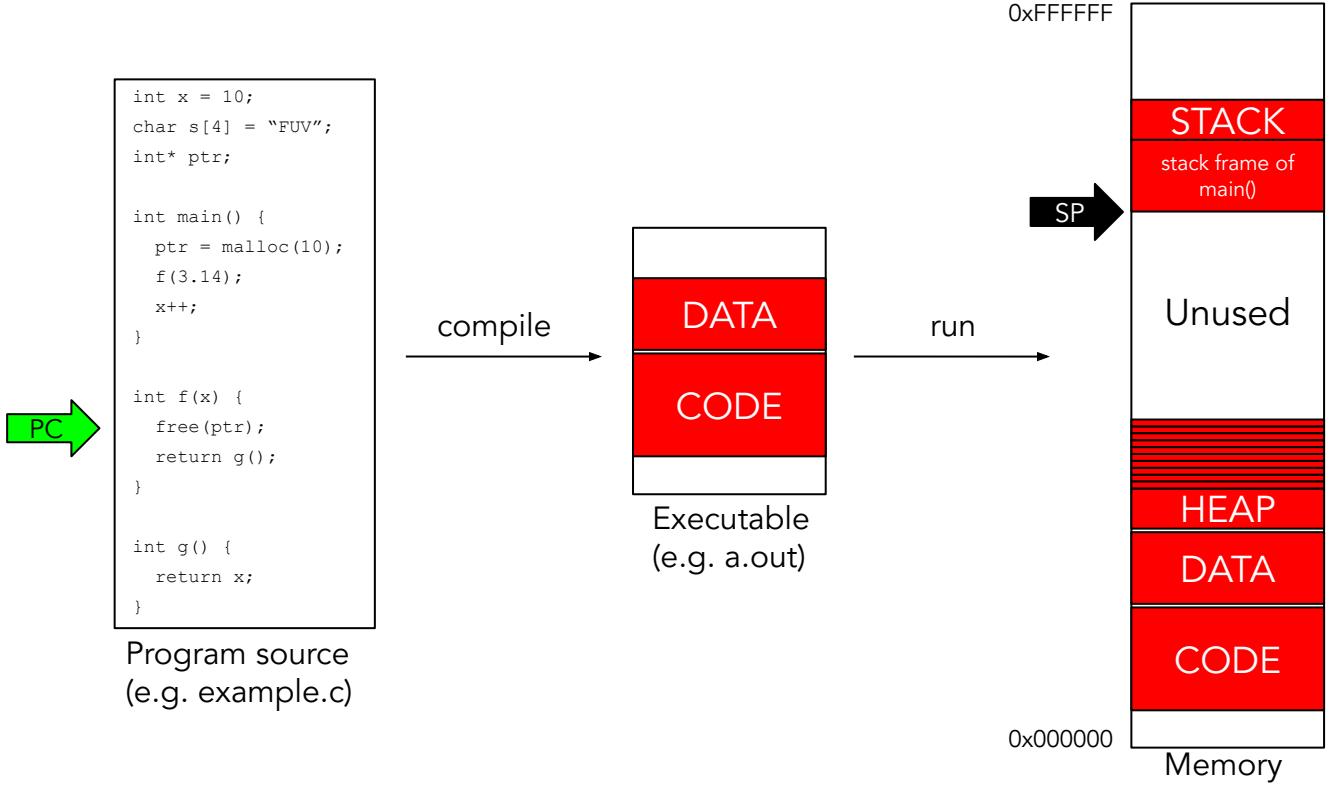
What happens when a program is compiled/run?



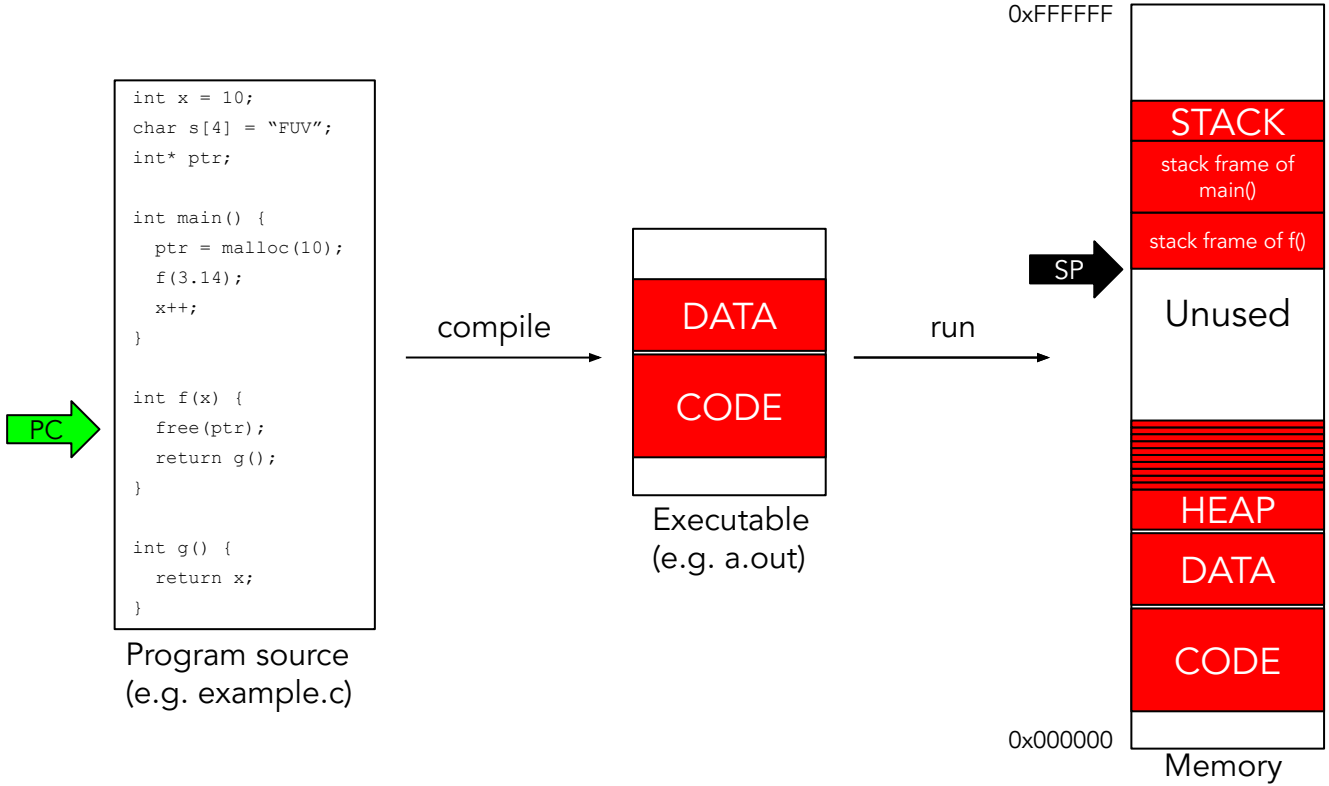
What happens when a program is compiled/run?



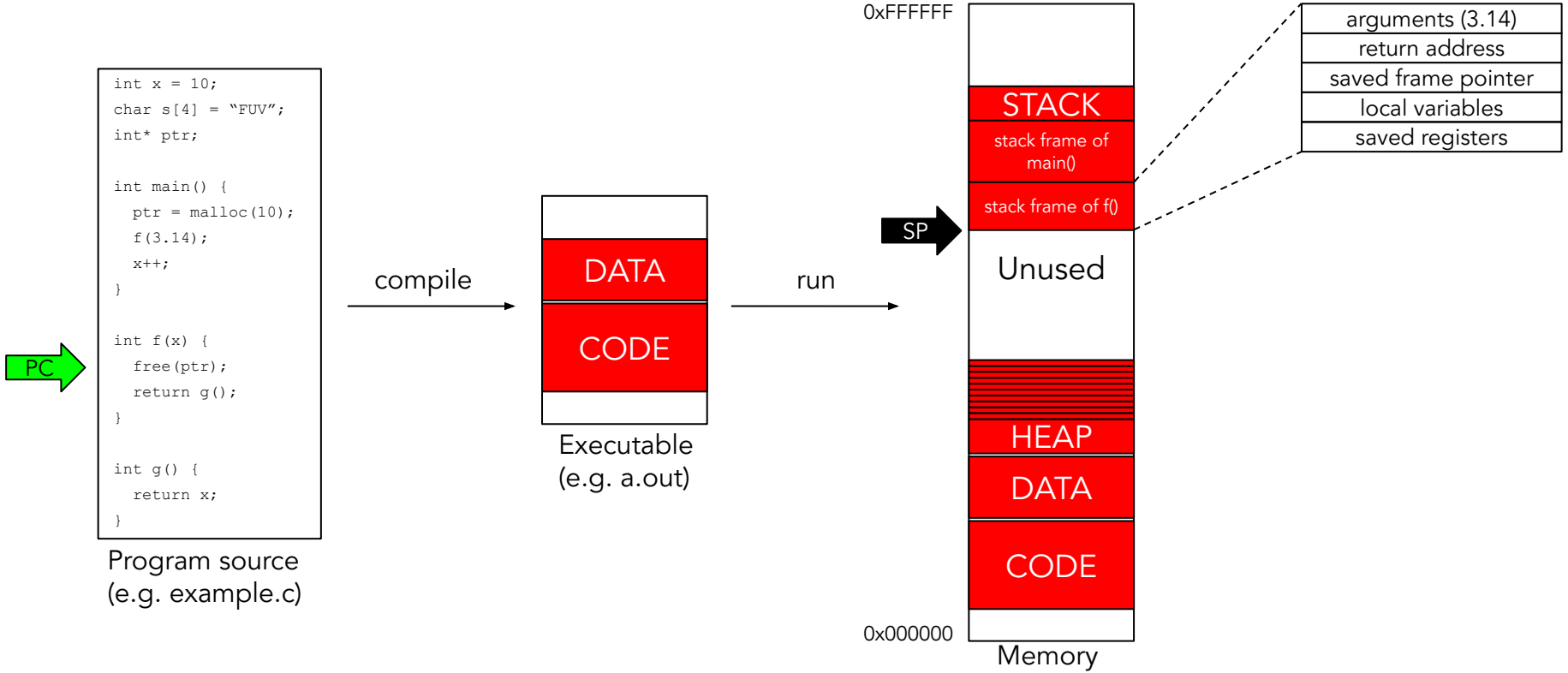
What happens when a program is compiled/run?



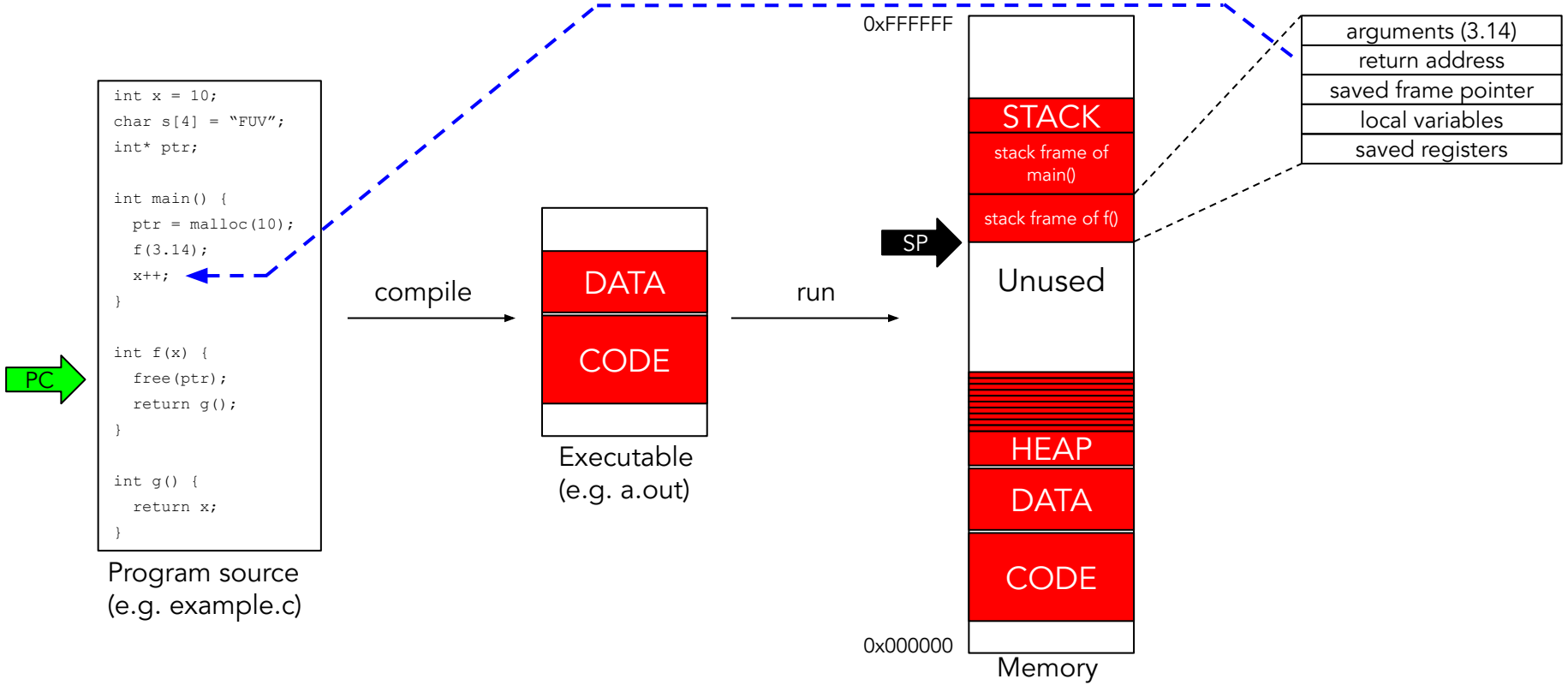
What happens when a program is compiled/run?



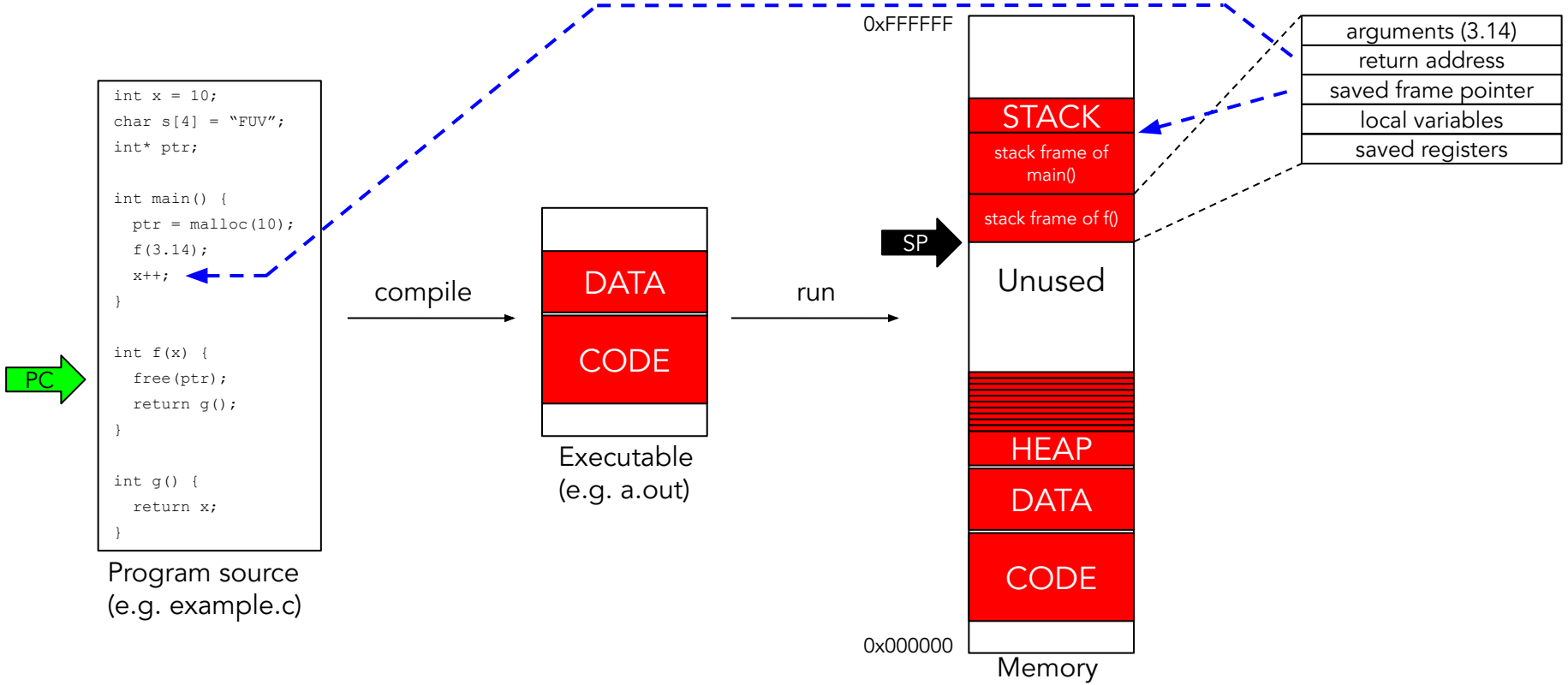
What happens when a program is compiled/run?



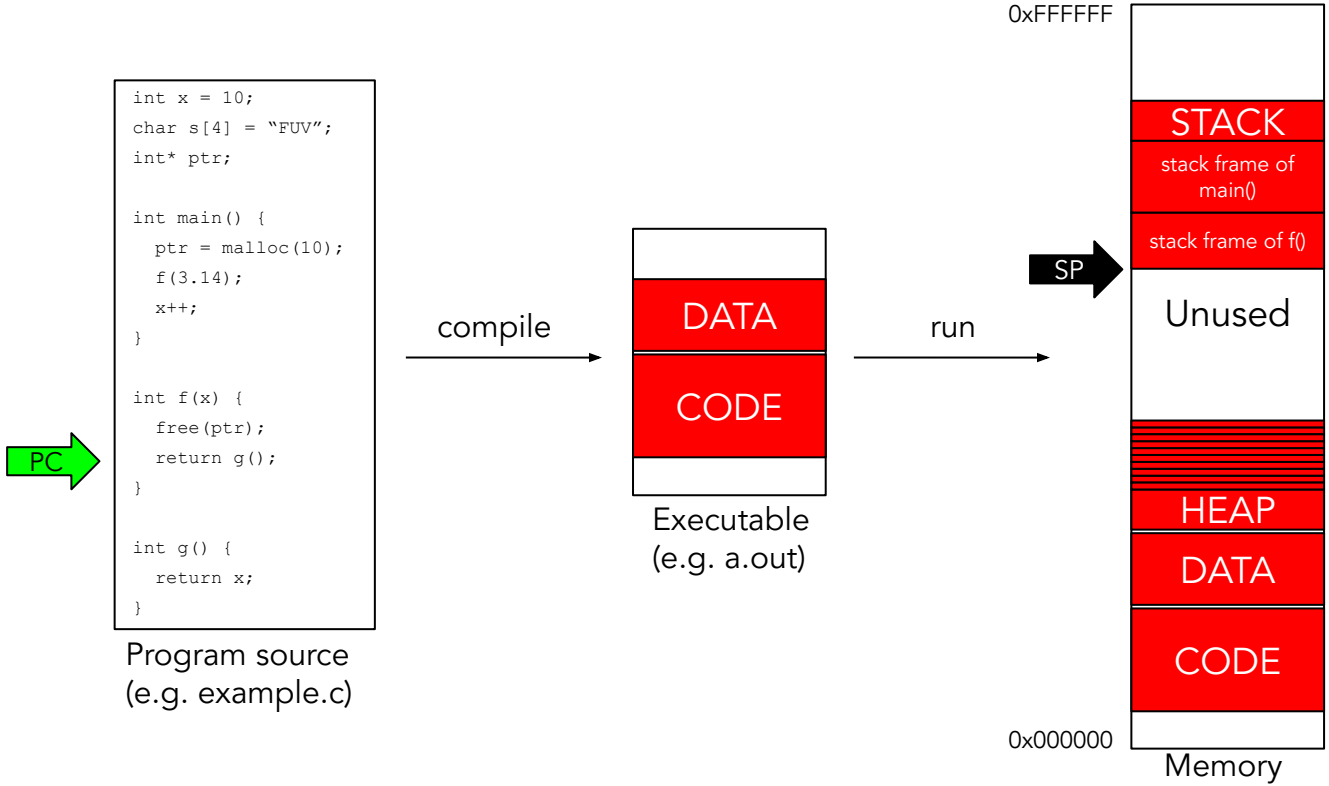
What happens when a program is compiled/run?



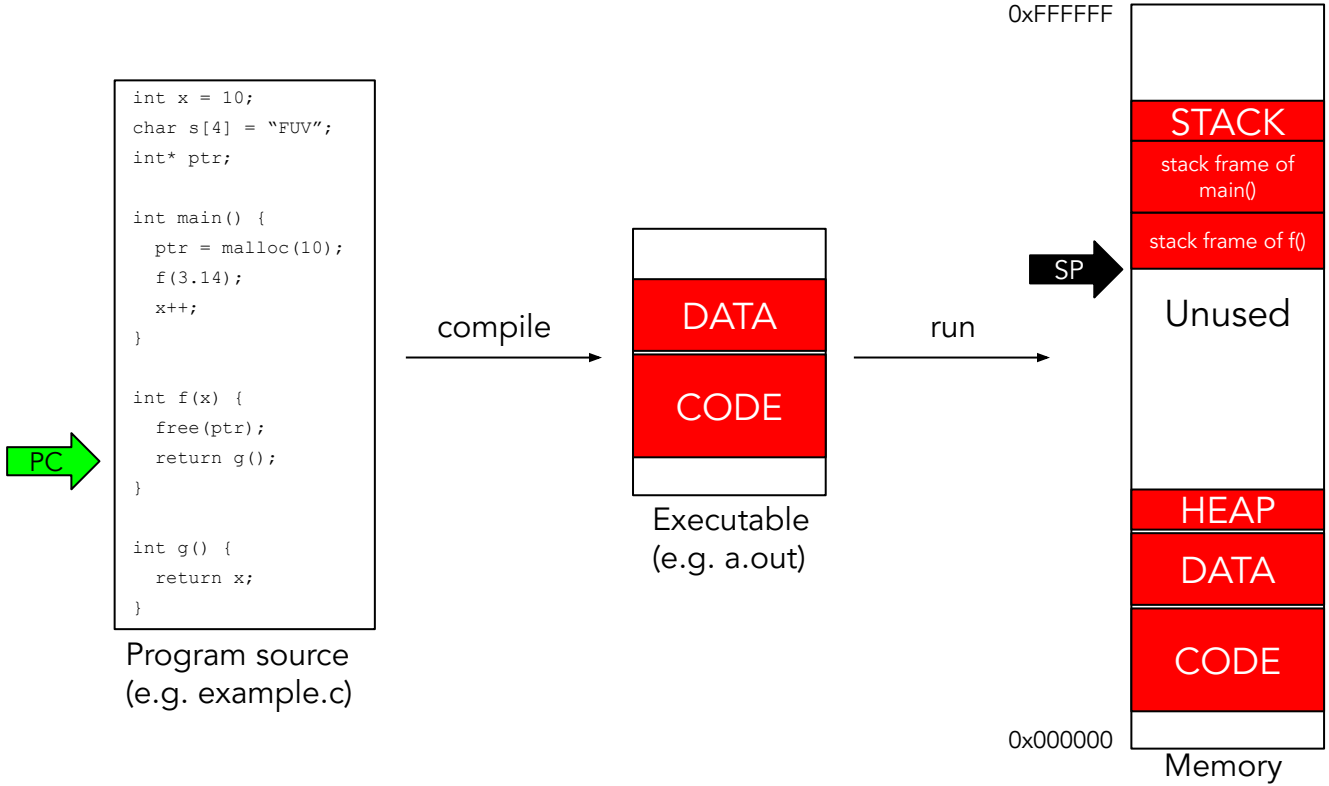
What happens when a program is compiled/run?



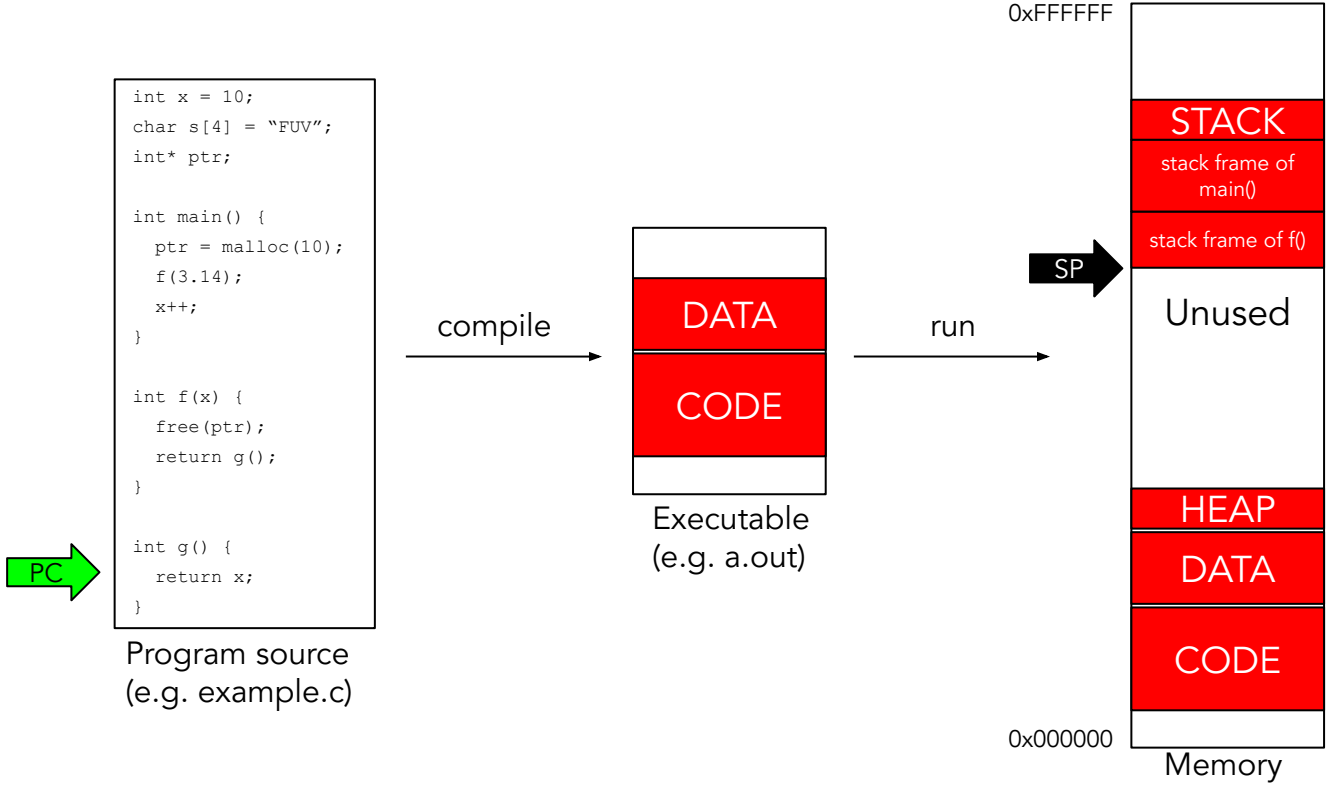
What happens when a program is compiled/run?



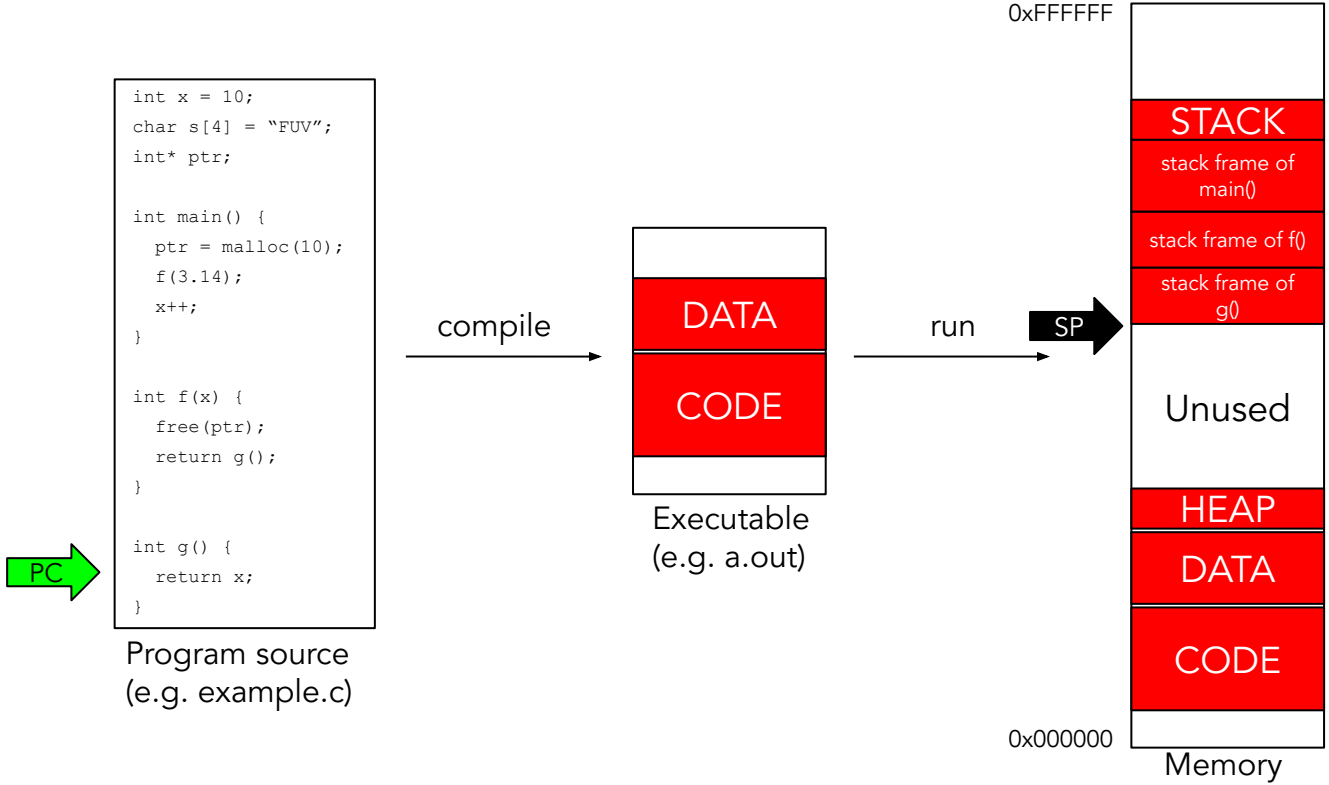
What happens when a program is compiled/run?



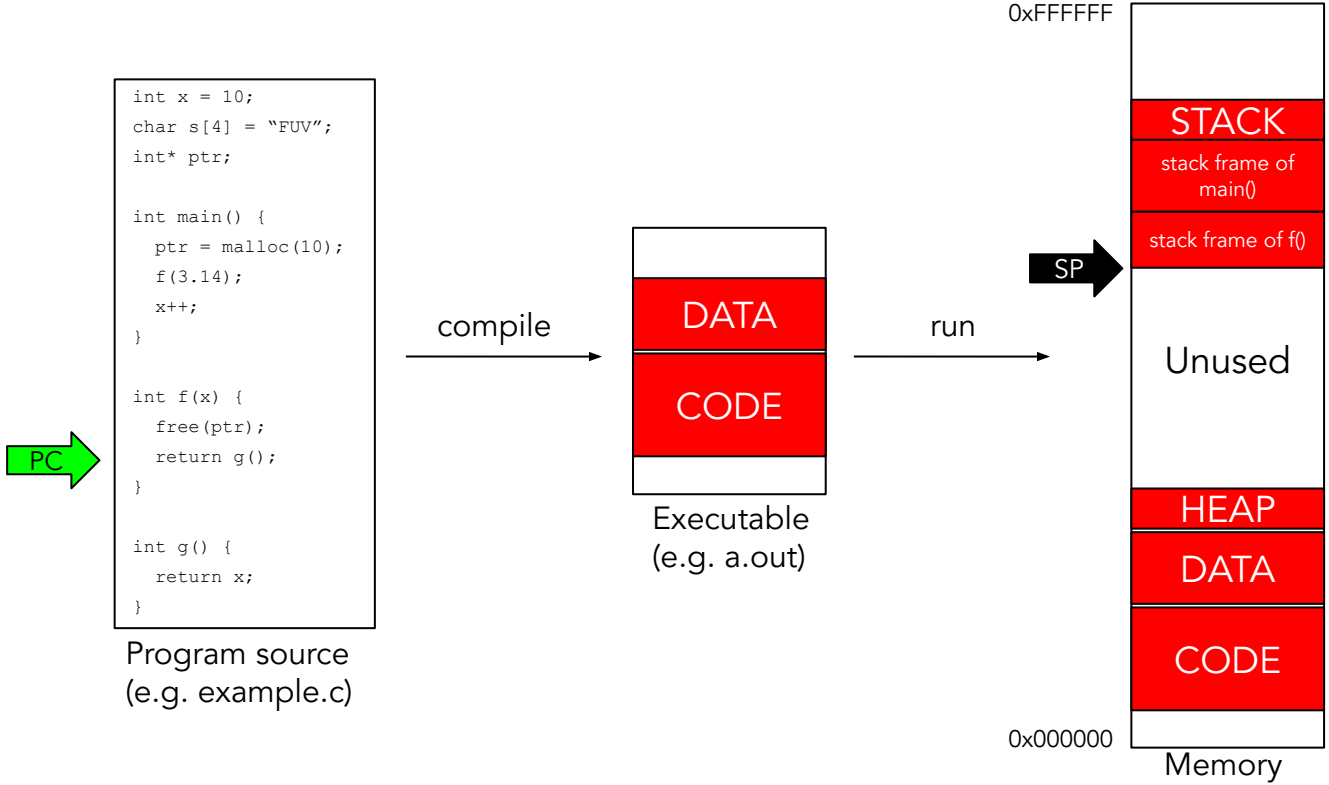
What happens when a program is compiled/run?



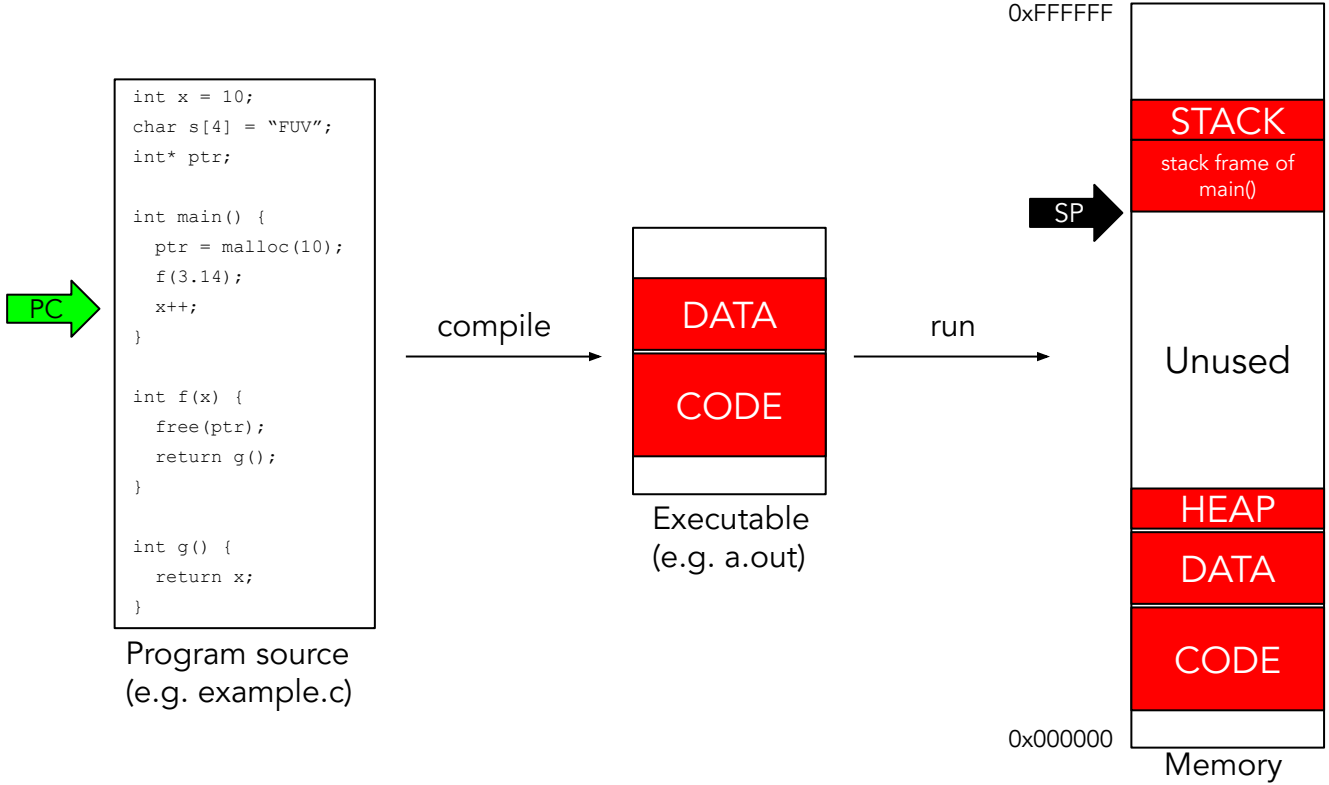
What happens when a program is compiled/run?



What happens when a program is compiled/run?



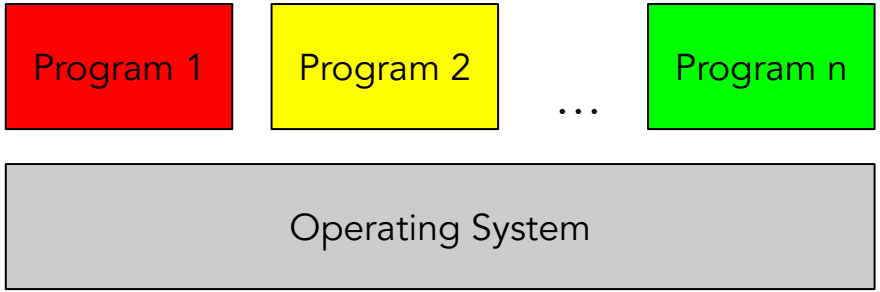
What happens when a program is compiled/run?

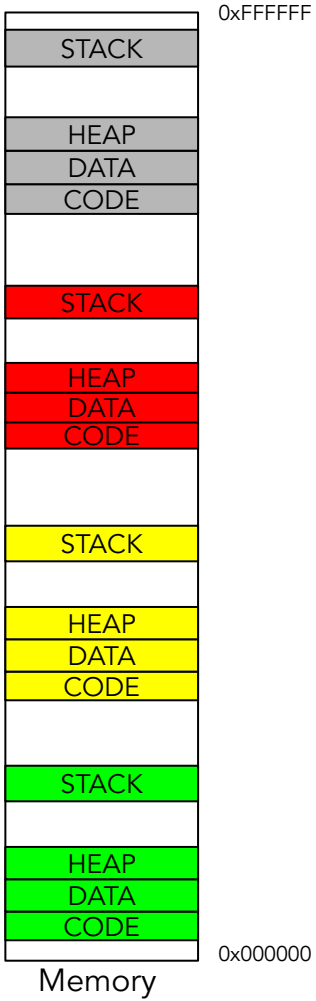


Processes

- Program vs Process
  - Process is an instance of a running program
  - “Running a program” = creating a process
  - Process is an abstraction of a computer

Multiprogramming





What can your program do?

What can your program do?

- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?

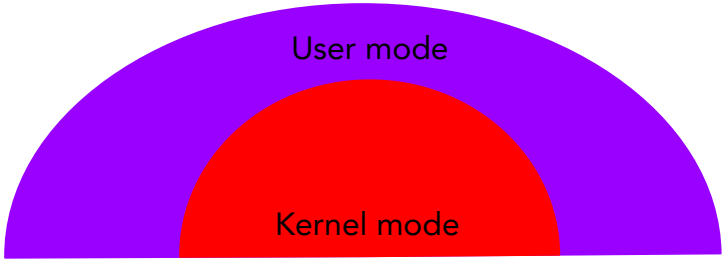
What can your program do?

- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?

What can your program do?

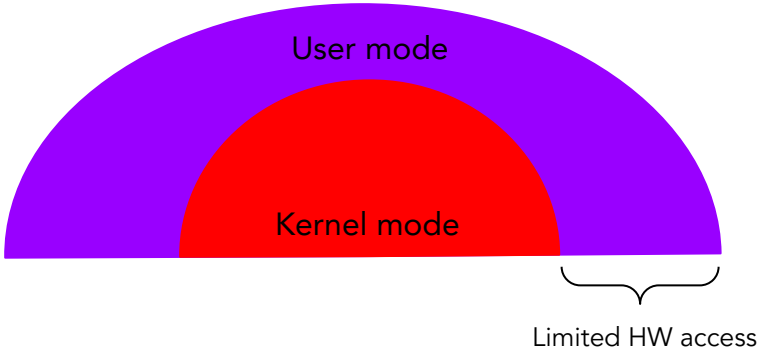
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



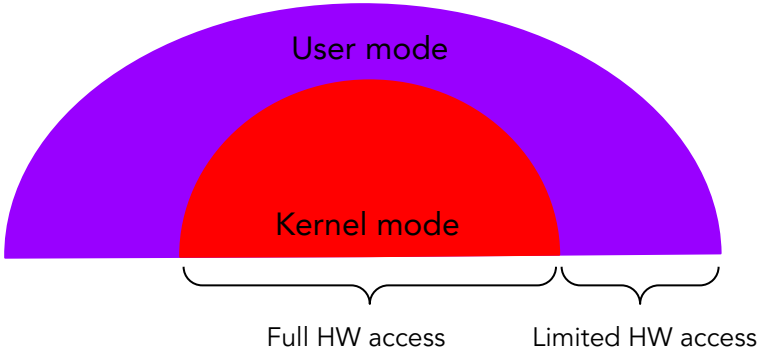
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



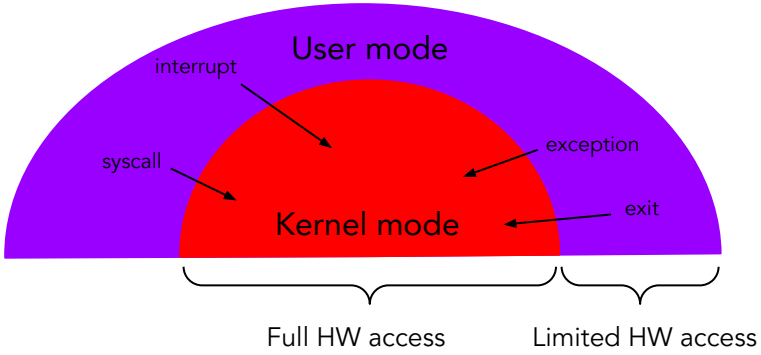
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



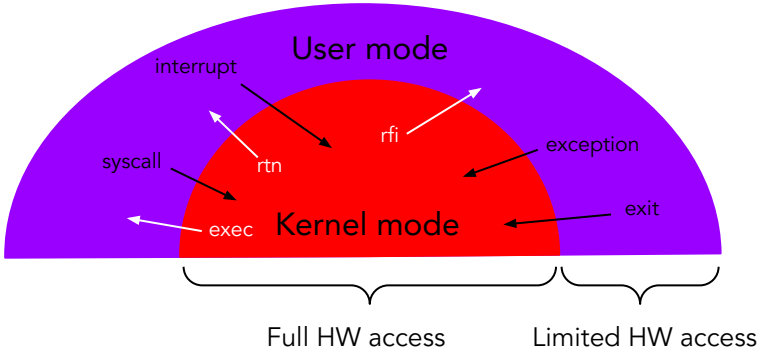
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



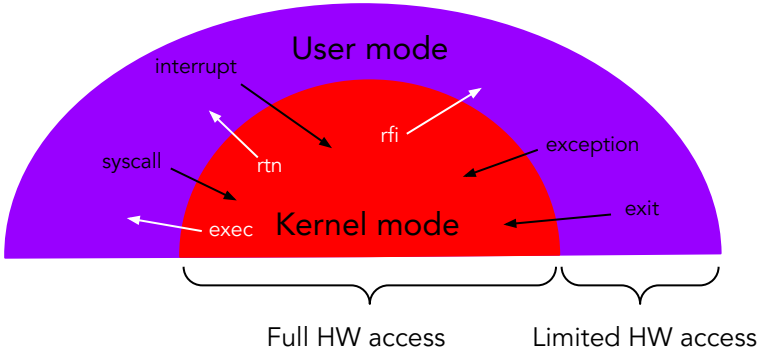
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



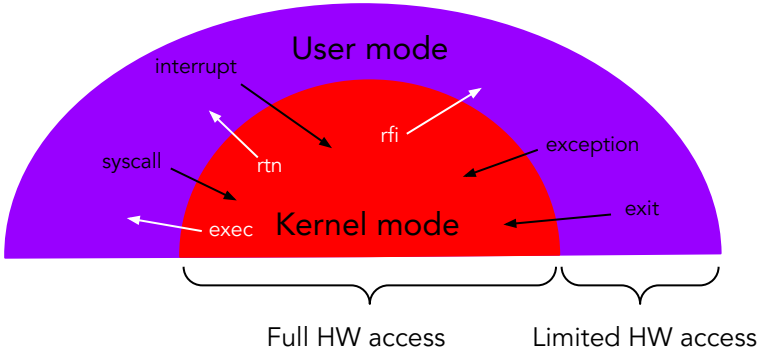
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode

What can your program do?



- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode
- So how can your program perform these tasks?

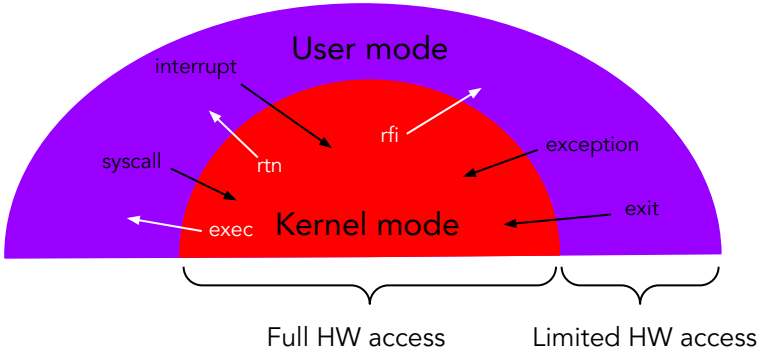
What can your program do?

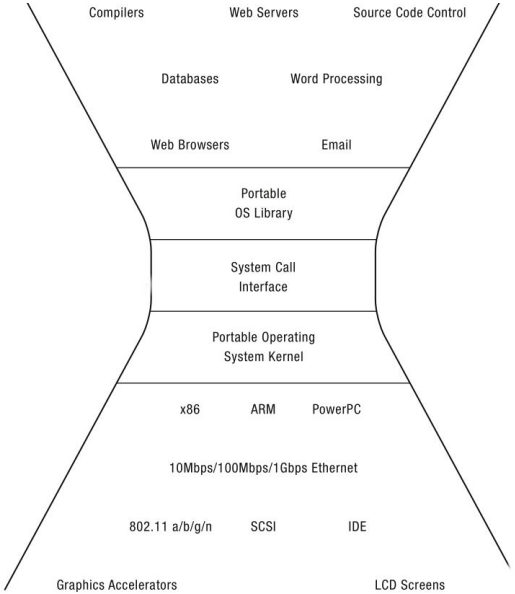


- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode
- So how can your program perform these tasks?
  - System calls

What can your program do?

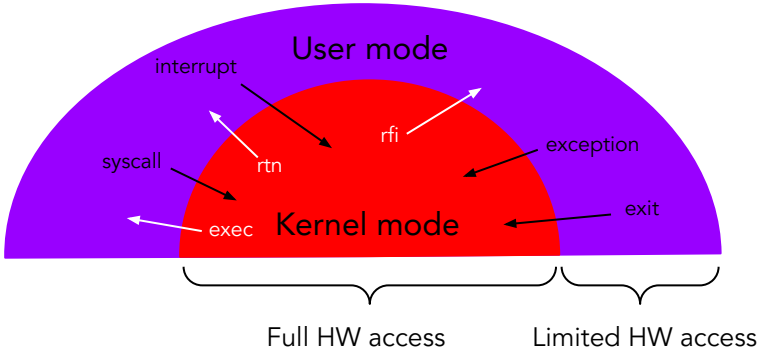
- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode
- So how can your program perform these tasks?
  - System calls

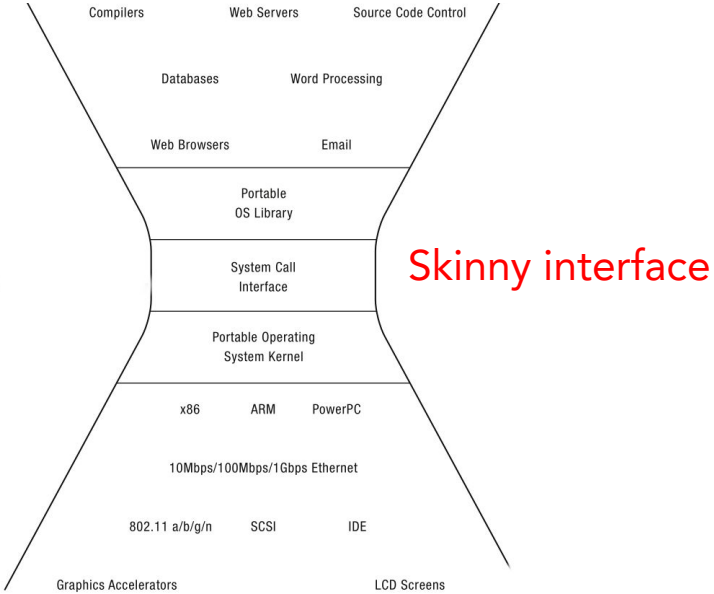




What can your program do?

- Can your program directly
  - Run a new program?
  - Terminate the current program?
  - Get the time or set alarms?
  - Read input/write to screen?
  - Create/read/write/delete files?
  - Send/receive network packets?
- No, why?
  - Privileged instructions (e.g. CLI, STI, LTR, etc) can be only executed in kernel mode
- So how can your program perform these tasks?
  - System calls

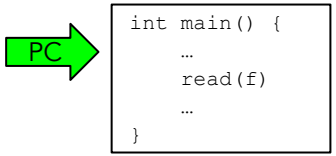




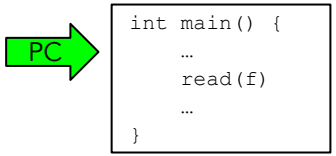
Executing a system call

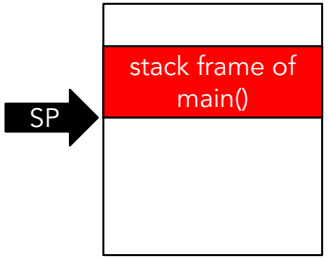
```
int main() {
    …
    read(f)
    …
}
```

Executing a system call

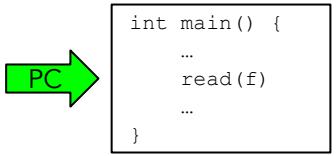


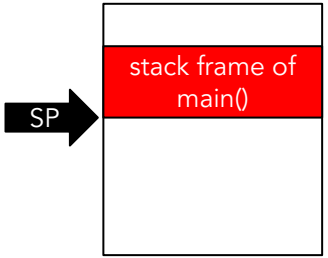
Executing a system call



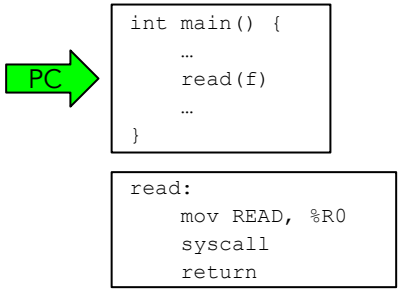


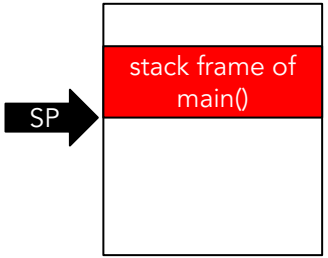
Executing a system call





Executing a system call



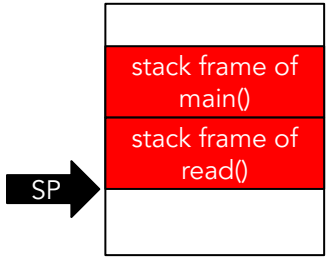


Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

PC

```
read:
    mov READ, %R0
    syscall
    return
```



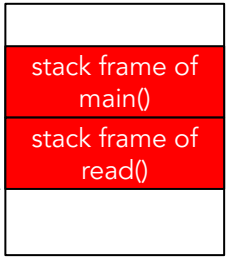
Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

PC

```
read:
    mov READ, %R0
    syscall
    return
```

SP



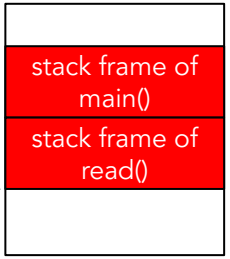
Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

PC

```
read:
    mov READ, %R0
    syscall
    return
```

SP



Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

PC

SP

stack frame of main()

stack frame of read()

User space

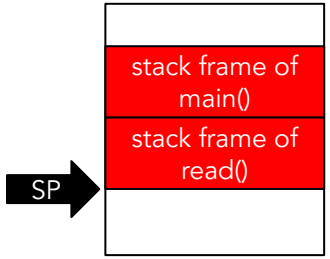
Kernel space

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

PC

```
read:
    mov READ, %R0
    syscall
    return
```



User space

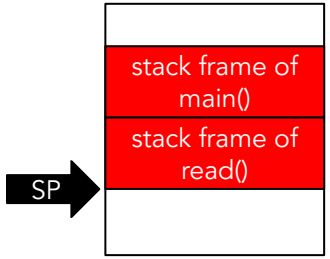
Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

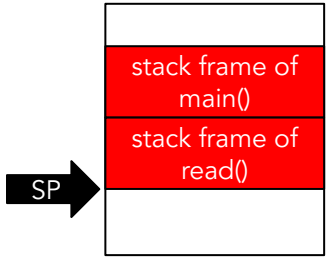
PC

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

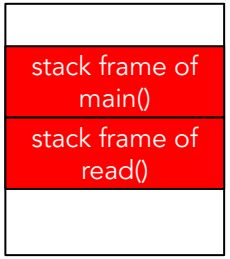
PC

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

PC

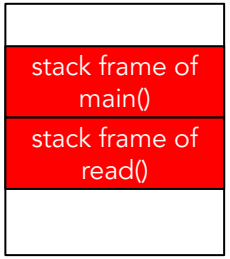
```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

SP

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

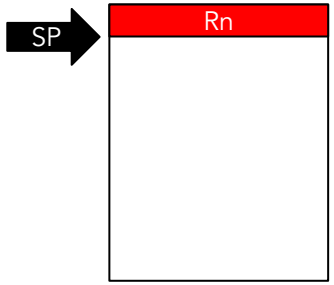
```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

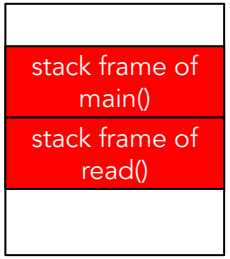
```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```



Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

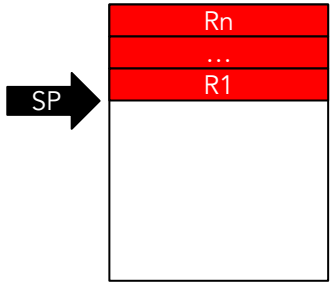
```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

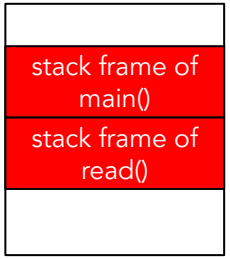
```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```



Executing a system call

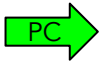
```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



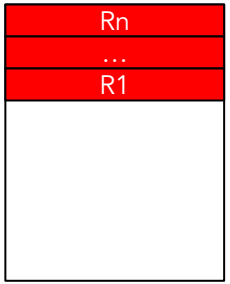
User space

Kernel space




```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

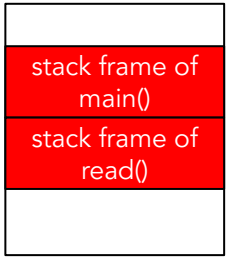
```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```



Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

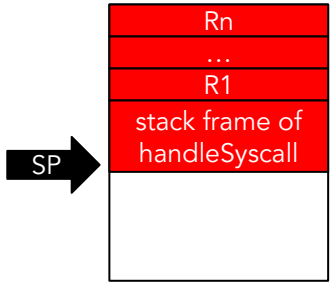
```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```



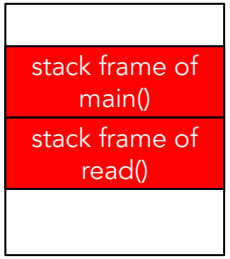
PC

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

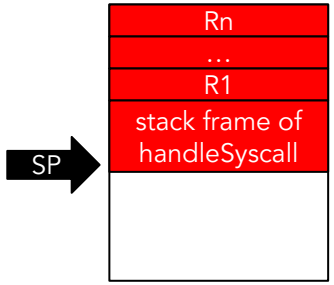
```
read:
    mov READ, %R0
    syscall
    return
```



User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```



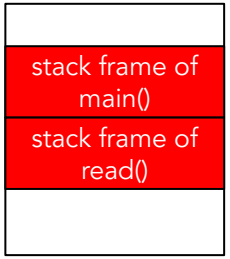
PC

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

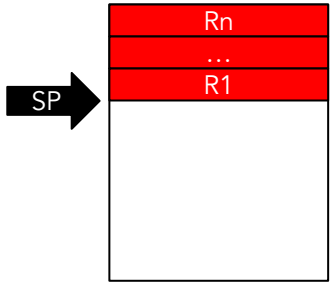


User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

PC

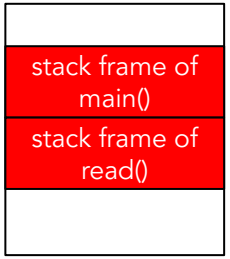



```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



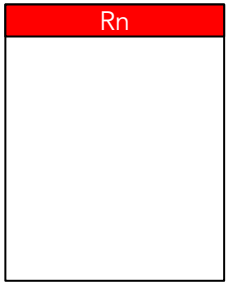
User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

PC

SP

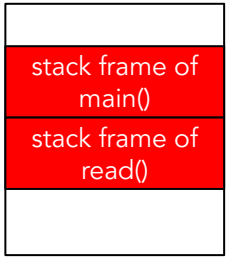



```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```



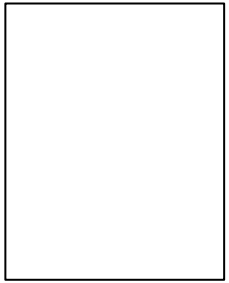
User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

PC

SP



```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

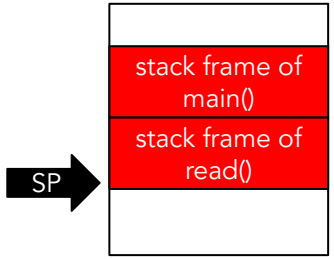
User space

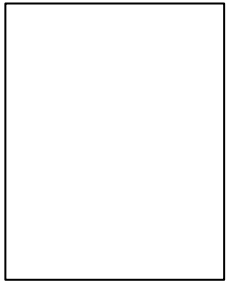
Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

PC

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```





Executing a system call

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

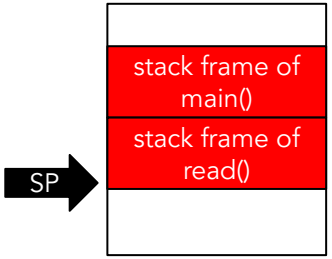
User space

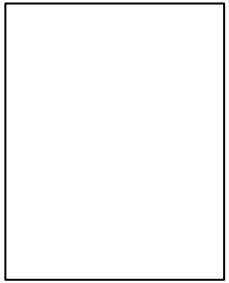
PC

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```





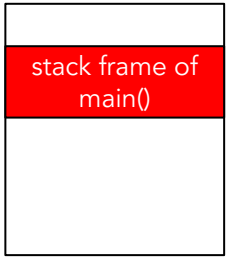
Executing a system call

PC

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

SP

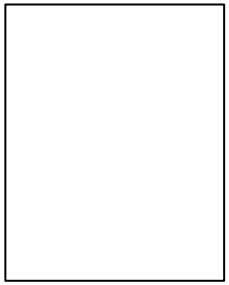


User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```



each process has 2 stacks

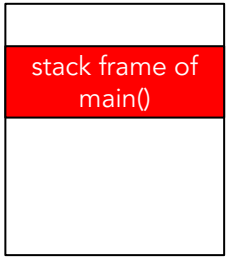
Executing a system call

PC

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

SP

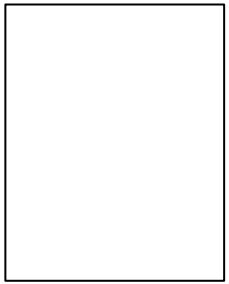


User space

Kernel space

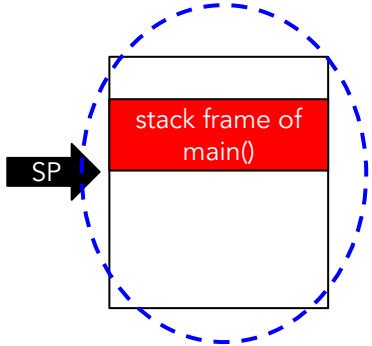
```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```



Executing a system call

each process has 2 stacks



user stack

PC

```
int main() {
    …
    read(f)
    …
}
```

```
read:
    mov READ, %R0
    syscall
    return
```

User space

Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```

Executing a system call

each process has 2 stacks

PC

```
int main() {
    …
    read(f)
    …
}
```

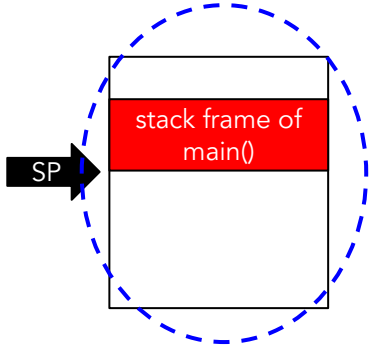
```
read:
    mov READ, %R0
    syscall
    return
```

User space

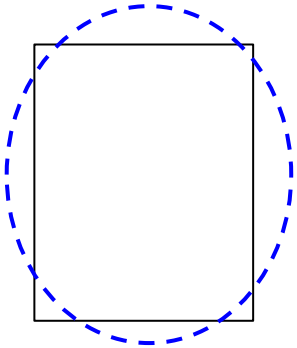
Kernel space

```
HandleIntrSyscall:
  mov %sp, $pcb
  mov $stacktop, $sp
  push %Rn
  …
  call handleSyscall
  …
  pop %Rn
  mov $pcb, %sp
  return_from_interrupt
```

```
int handleSyscall (int type) {
  switch (type) {
    case READ: …
  }
}
```



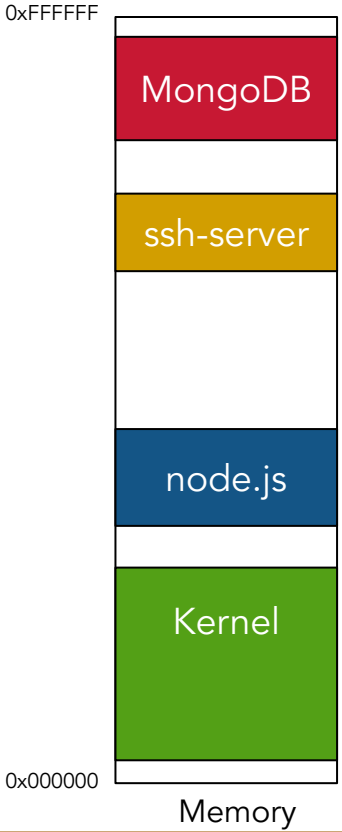
user stack



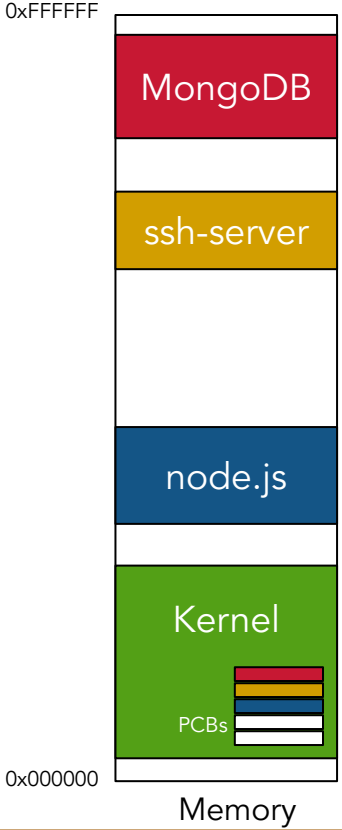
interrupt stack

How does the OS manage processes?

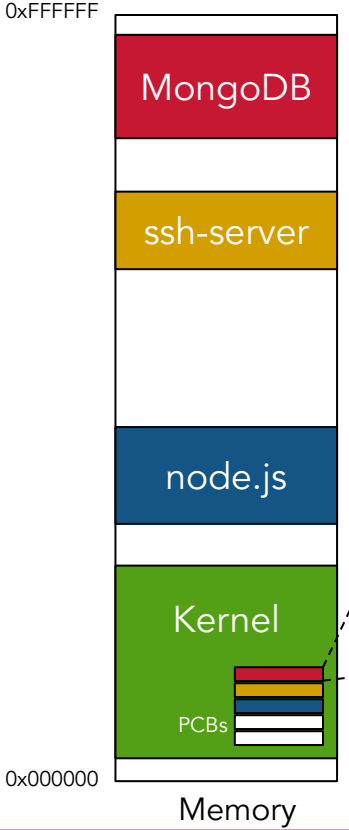
How does the OS manage processes?



How does the OS manage processes?



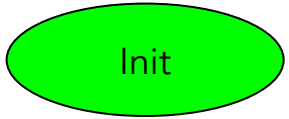
How does the OS manage processes?



Process control block (PCB)

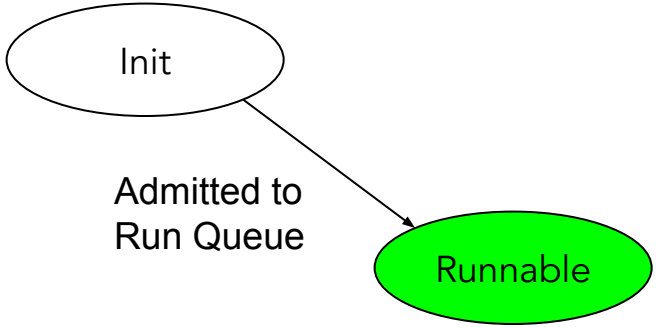
- location in memory (page table)
- location of executable on disk
- which user is executing this process (uid)
- process identifier (pid)
- process status (running, waiting, finished, etc.)
- scheduling information
- interrupt stack
- saved user SP points into user stack
- saved kernel SP points into interrupt stack
- …

Process life cycle



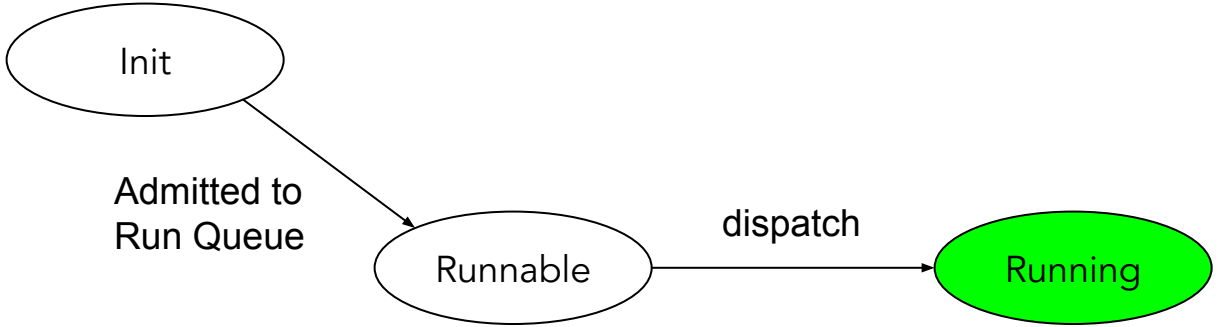
PCB: being created
Registers: uninitialized

Process life cycle



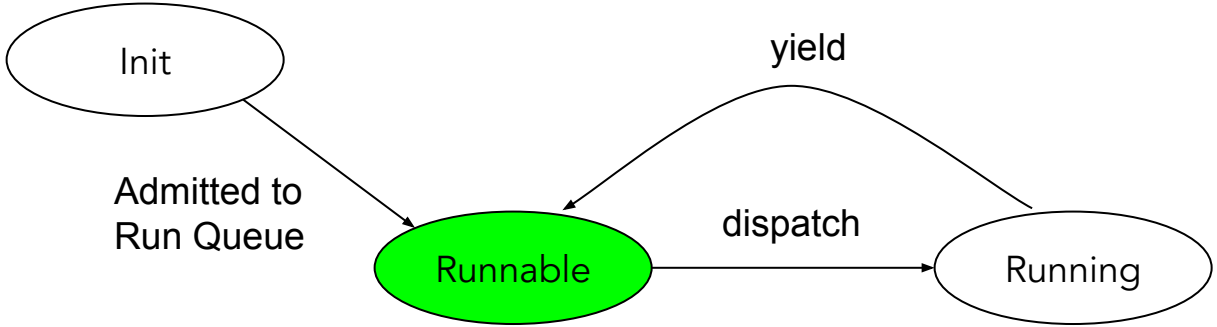
PCB: on Run Queue
Registers: push onto interrupt stack (by kernel)

Process life cycle



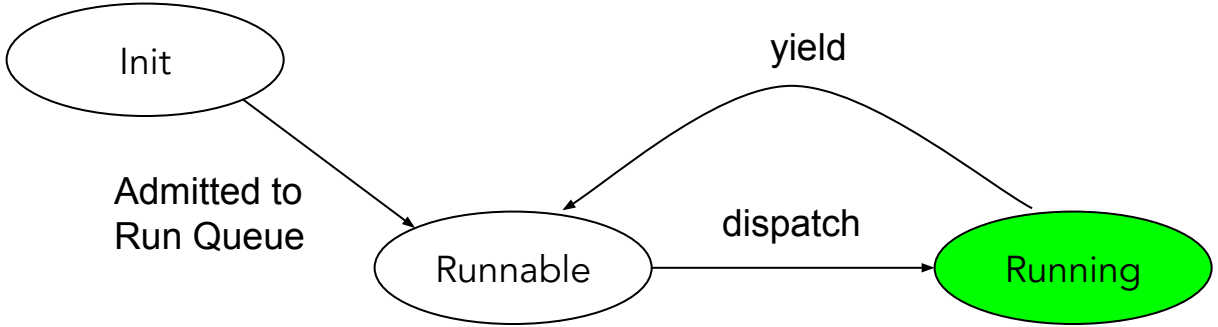
PCB: executing
Registers: popped from interrupt stack into CPU

Process life cycle



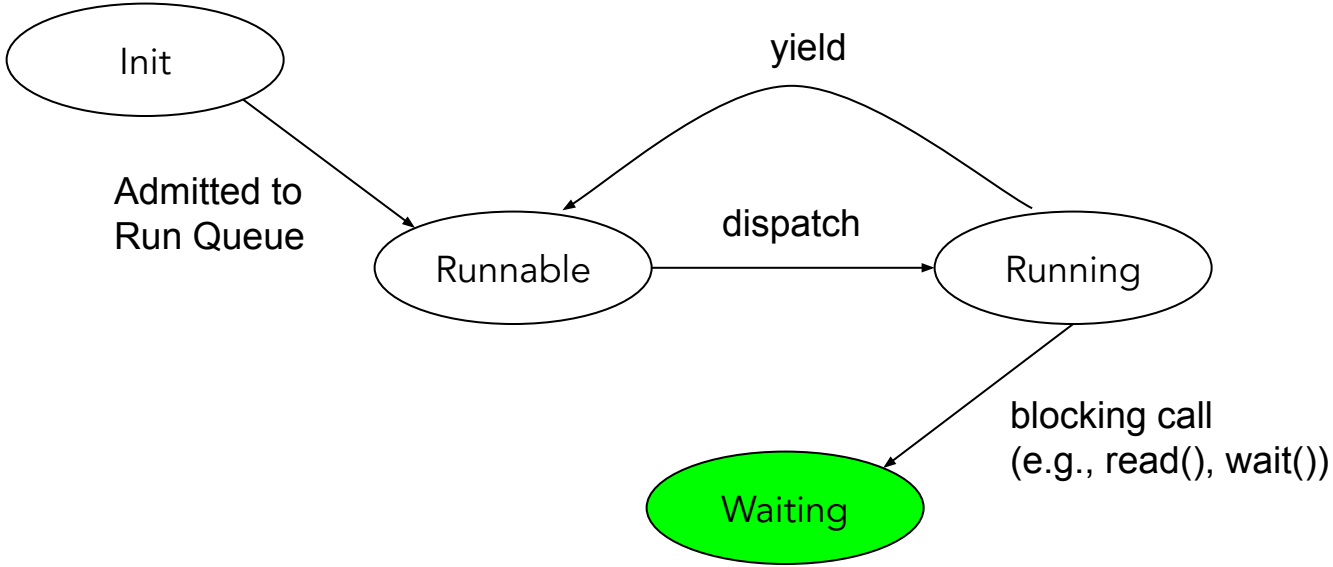
PCB: on Run Queue
Registers: push onto interrupt stack (by kernel)

Process life cycle



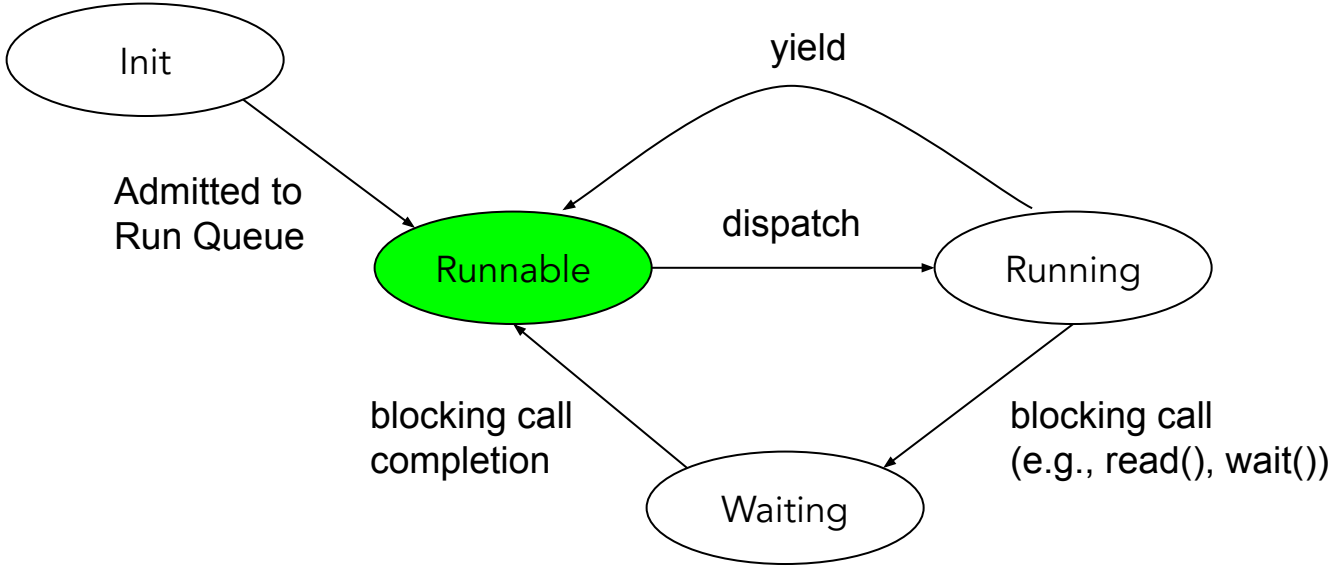
PCB: executing
Registers: restored from stack

Process life cycle



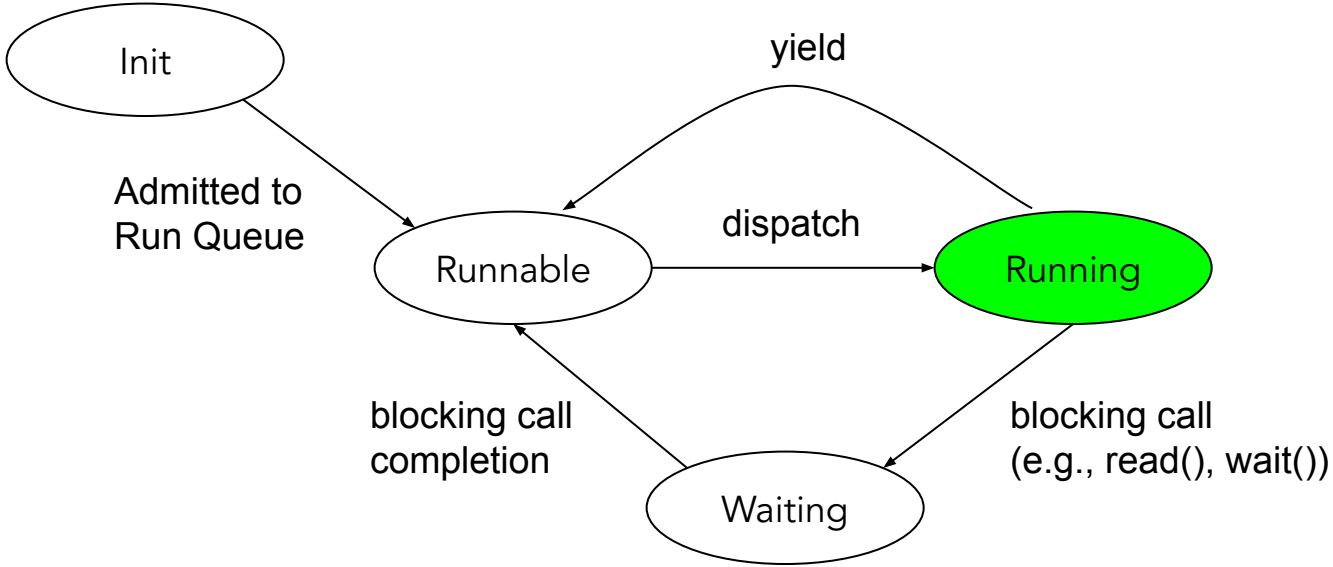
PCB: on specific waiting queue (file, timer, sockets …)
Registers: on interrupt stack

Process life cycle



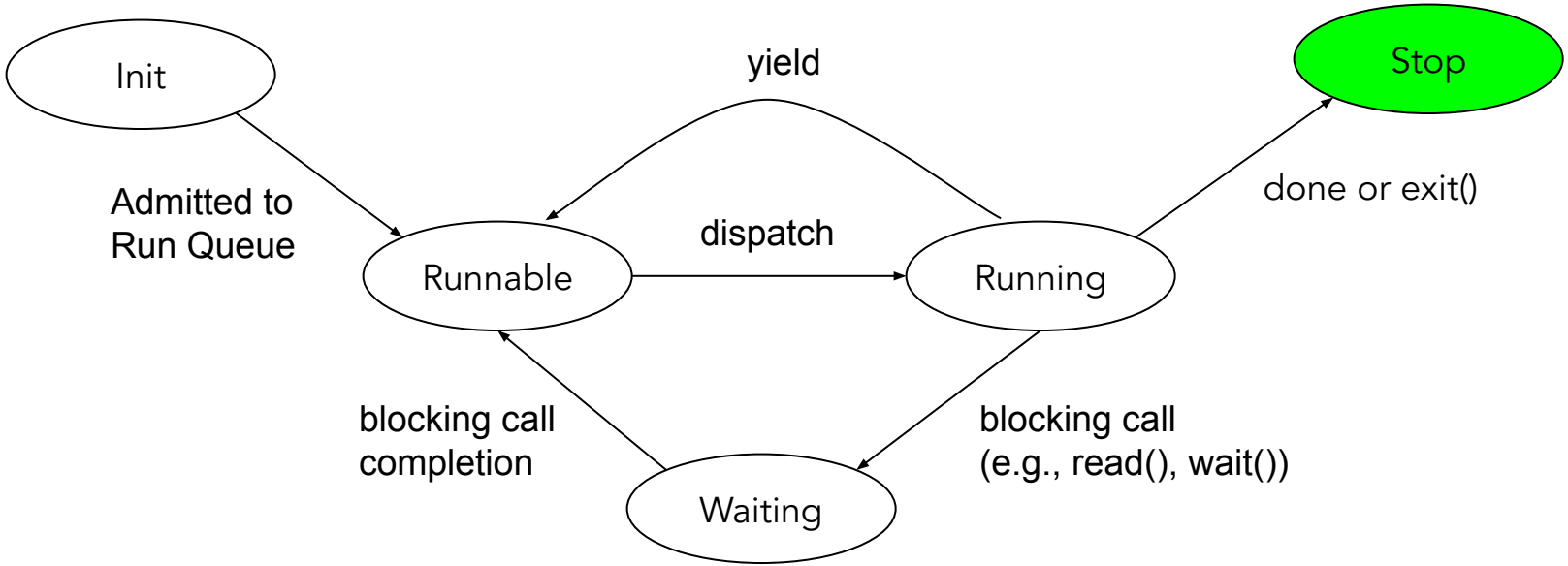
PCB: on Run Queue
Registers: push onto interrupt stack (by kernel)

Process life cycle



PCB: executing
Registers: restored from stack

Process life cycle



PCB: on Finished queue, then deleted
Registers: no longer needed

Zombie

- A finished process but its PCB has not been deleted
  - To report to its parent process
- A child process always
  - first becomes a zombie
  - then its PCB is removed
- The parent process
  - reads the exit status of the child process
  - reaps off the child process PCB.

How to clean zombie?

- Process cannot clean up itself
  - WHY NOT?
- Process can be cleaned up
  - either by any other process
    - check for zombies just before returning to RUNNING state
  - or by parent when it waits for it
    - but what if the parent dies first?
  - or by dedicated “reaper” process
- Linux
  - parent cleans up child process when waiting
  - if parent dies before child, child process is inherited by the initial process, which is continually waiting

Invariants of processes

- At most 1 process is RUNNING at any time (per core)
  - When CPU is in user mode, current process is RUNNING and its interrupt stack is empty
- If process is RUNNING
  - its PCB is not on any queue
- If process is RUNNABLE or WAITING
  - its interrupt stack is non-empty
  - its PCB is either
    - on the run queue (if RUNNABLE)
    - on some wait queue (if WAITING)
- If process is FINISHED
  - its PCB is on finished queue

Operations on processes

- Process creation
  - fork()
    - Creates a child process as a clone of the current process
    - Returns child pid to parent process, 0 to child process
  - exec(prog, args)
    - Run the application prog in the current process with the specified arguments (replacing any code and data that was in the process already)
  - createProcess(...) = fork() + exec(...)
    - only in Windows
  - wait(&status)
    - Pause until a child process has exited
- Process termination
  - exit(status)
    - Tell the kernel the current process is complete and should be garbage collected
  - kill(pid, type)
    - Send an interrupt of a specified type to a process

Example

```c
#include <stdio.h>
#include <unistd.h>

int main() {
   int child_pid = fork();

   if (child_pid == 0) {      // child process
        printf("I am process %d\n", getpid());
   }
   else {                     // parent process.
        printf("I am the parent of process %d\n", child_pid);
   }
   return 0;
}
```

What is printed out?

Summary

1. What is a process?
   - an instance of a running program
   - an abstraction of a computer
2. What is the difference between the user processes vs the kernel?
   - User mode vs kernel mode
3. How does the operating system manage processes?
   - PCB
4. How can we create and manipulate processes?
   - fork, exe, wait, exit, kill